Generating Music with Machine Learning

UE24CS352A Machine Learning Mini-Project — Group 22 — Problem 22
Nagashree R Patil (PES1UG24CS289), Nallamalli Kanaka Mani Sai Akhil (PES1UG24CS290)
DRAFT: text in red must be replaced with your actual results after running the notebook.

1. Problem Statement

Music composition is a sequence-modelling problem: most pieces reuse and vary patterns that appeared before. Given MIDI recordings of piano music, the task is to learn a model that predicts the next musical event from the preceding ones and then use it to generate new pieces. We follow the setup of the Stanford project *Generating Music with Machine Learning* (Kang, Kim and Ringdahl), which compared a probabilistic chord-to-note model with recurrent neural networks, and we compare a Markov baseline against LSTM and GRU models on the same representation.

2. Dataset

We use the **MAESTRO v3.0.0** piano MIDI collection (Google Magenta), restricted to pieces by Chopin, similar to the Chopin etudes used in the reference paper. [N] files were used, giving [N] note events. Files are split **by piece** into train/validation/test (80/10/10: [a/b/c] pieces) so that no piece appears in more than one split. The vocabulary is built from the training set only and contains the 1,500 most frequent events; all other events map to an `<unk>` token ([x]% of test events), which is ignored in the loss and in sampling.

3. Approach

Representation. Note start times are quantised to a 0.125 s grid. All notes starting at the same time form one *event* (we keep the four highest pitches), paired with the gap to the next onset (1 to 16 grid steps). A song is therefore a sequence of (chord, duration) tokens, similar to the "new note event" organisation that worked best for the LSTM in the reference paper. Training samples are windows of 64 consecutive events, with the target being the same window shifted by one event.

Models. (i) A *Markov bigram baseline* that predicts the most frequent successor of the previous event (in the spirit of the paper's Naive-Bayes-like model). (ii) A 2-layer *LSTM* and (iii) a 2-layer *GRU*, each with a 128-d embedding, 256 hidden units, dropout 0.3 and a softmax over the vocabulary. The paper's vanilla network overfitted badly (5% train vs 88% test error), so we use per-piece splitting, dropout and early stopping on validation loss.

Evaluation. Cross-entropy on held-out pieces, perplexity (exp of the loss), and top-1/top-5 next-event accuracy. Generated music is also assessed with a short listening survey in which participants label clips as human or computer composed, as in the paper's music Turing test.

4. Implementation Overview

The project is a single Colab notebook in Python using *pretty_midi* for MIDI parsing and writing, *PyTorch* for the models, and *pandas/matplotlib* for results. The pipeline is: download and parse MIDI → build events and vocabulary → create windowed datasets → train LSTM and GRU (Adam, learning rate 2e-3, batch size 128, gradient clipping at 1.0, early stopping with patience 4) → evaluate on the test set → generate music. Generation seeds the model with the first 32 events of a held-out piece, then samples 300 events using temperature 0.9 and top-15 filtering, and writes a .mid file. Results, plots and generated clips are saved in the repository. Training used a Colab [GPU type] and took [minutes] per model.

Table 1: Test-set results (survey column to be filled after the listening survey)

Model	Test loss	Perplexity	Top-1 acc.	Top-5 acc.	Survey: % judged human
Markov (bigram)	n/a	n/a	4.0%	26.5%	[XX]%
LSTM	3.401	30.0	32.9%	59.5%	[XX]%
GRU	3.498	33.1	32.2%	58.4%	[XX]%

5. Conclusions

The LSTM performed best, with a test perplexity of 30.0 (loss 3.401) and top-1/top-5 accuracy of 32.9%/59.5%, narrowly ahead of the GRU (perplexity 33.1; 32.2%/58.4%). Both recurrent models are far better than the Markov bigram baseline (4.0%/26.5%), which shows that longer musical context matters; for scale, guessing uniformly among 1,500 events would give a perplexity near 1,500. The LSTM-GRU gap is small and comes from a single run, so we do not claim it is significant. [Add: train vs validation loss gap from loss_curves.png, how the clips sounded, and the survey result.]

Limitations. The vocabulary is capped at 1,500 events and chords at four notes, so rare and dense passages are simplified; note velocity and dynamics are not modelled, which the paper noted makes generated music sound less human; and the survey sample is small. **Future work:** memory-focused models such as a Transformer, velocity-aware inputs, and separate pitch and duration outputs.

References

[1] D. Kang, J. Y. Kim, S. Ringdahl, *Project milestone: Generating music with machine learning*, Stanford, 2018. [2] C. Hawthorne et al., *Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset*, ICLR 2019. [3] C. Raffel and D. Ellis, *Intuitive Analysis, Creation and Manipulation of MIDI Data with pretty_midi*, 2014.